

RECITATION 8: DNA GENE TEST

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Problem Statement

You have a set of n DNA samples. You want to see if there is a variant of a particular gene that exists in a majority of the n samples. (We simplify this to say a gene is a particular location on a chromosome.) Because of how DNA has lots of variants, all you are able to do is run a test, $GeneTest(g1, g2)$, on two samples, and the test returns true if the two samples have the same gene variant on the chromosome in question and false if they have a different gene variant.

Note: this is not telling you what variant to look for, just whether there is some variant on that gene that exists in a majority of the population. Also note that because we are testing a single gene, there can be at most one “majority variant” in a population.

Design an algorithm that determines if there is a gene variant that exists in a majority ($50\% + 1$) of the n samples. Because DNA tests are still expensive to perform, you want an algorithm that minimizes the total number of tests needed.

Example:

Note: This may be a simplification of certain terms relating to biology. This serves as a simple example.

Input: S: [Variant A, Variant A, Variant B, Variant A, Variant C, Variant A, Variant B]

Output: Majority Variant: A

Solution

Pseudocode:

Note that, since we are not able to “read out” the variants, the algorithm uses individuals to represent variants. For this reason, MAJORITYVARIANT returns an individual who harbors the majority variant in the respective (sub)group. Similarly, COUNT takes as input an individual in addition to a (sub)group and counts the variant harbored by that individual in the subgroup.

Algorithm 1 Majority Variant Detection

```
1: procedure MAJORITYVARIANT( $S$ )
2:   if length( $S$ ) = 1 then
3:     return  $S[1]$ 
4:   end if
5:
6:    $mid \leftarrow \text{length}(S)/2$ 
7:   leftMajor  $\leftarrow$  MAJORITYVARIANT( $S[:mid]$ )
8:   rightMajor  $\leftarrow$  MAJORITYVARIANT( $S[mid:]$ )
9:
10:  if ARETHEYTHESAME(leftMajor, rightMajor) then
11:    return leftMajor
12:  else
13:    leftCount  $\leftarrow$  COUNT( $S$ , leftMajor)
14:    rightCount  $\leftarrow$  COUNT( $S$ , rightMajor)
15:
16:    if leftCount > length( $S$ )/2 then
17:      return leftMajor
18:    else if rightCount > length( $S$ )/2 then
19:      return rightMajor
20:    else
21:      return None
22:    end if
23:  end if
24: end procedure
25:
26: function COUNT( $S$ ,  $x$ )
27:    $count \leftarrow 0$ 
28:   for  $y$  in  $S$  do
29:     if ARETHEYTHESAME( $x$ ,  $y$ ) then
30:        $count \leftarrow count + 1$ 
31:     end if
32:   end for
33:   return  $count$ 
34: end function
```

Runtime Analysis:

The procedure COUNT makes n calls to ARETHEYTHESAME when it is called for a population of size n . Let $T(n)$ denote the runtime of MAJORITYVARIANT on a population of size n . Such an instance calls procedure COUNT twice on instances of size $n/2$ and it calls MAJORITYVARIANT recursively on two instances of size $n/2$. Thus the recurrence is $T(n) = 2T(n/2) + \Theta(n)$. Using the Master Theorem, we can see that $a = 2$ and $b = 2$, and $f(n) = n$. Since $f(n) = n = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n)$, we can conclude it is Case 2.

$$T(n) = \Theta(n \lg n)$$

Proof of correctness

Now we prove that the majority of a set must be the majority of one of the halves of the set. Let's use a proof by contradiction, while also using induction:

Proof. We will use induction to prove the correctness.

Base case: For $n=1$, the algorithm will return the one individual, which is trivially the majority individual.

Induction: Assume for some $\text{len}(S) = k + 1$, the algorithm will return an individual with the majority variant or no majority individual on any sub-problem $\text{len}(S) = c$ for $c < k + 1$.

If one of the returned individuals has one of the majority variants, we are done. However, suppose neither of the returned individuals has the majority variant. Let's prove this is impossible.

Since the majority variant is not a majority in the first half, that must mean that it can occur at most $\frac{1}{2} \lfloor \frac{k+1}{2} \rfloor$ times in the first half. Similarly, since the majority variant is not a majority in the second half, that must mean it can occur at most $\frac{1}{2} \lceil \frac{k+1}{2} \rceil$ times in the second half.

Adding these together, this becomes:

$$\begin{aligned} \frac{1}{2} \left\lfloor \frac{k+1}{2} \right\rfloor + \frac{1}{2} \left\lceil \frac{k+1}{2} \right\rceil &= \frac{1}{2} \left(\left\lfloor \frac{k+1}{2} \right\rfloor + \left\lceil \frac{k+1}{2} \right\rceil \right) \\ &= \frac{1}{2} \left(\left\lfloor \frac{k+1}{2} \right\rfloor + \left\lfloor \frac{k+1+2-1}{2} \right\rfloor \right) = \frac{1}{2} \left(\left\lfloor \frac{k+1}{2} \right\rfloor + \left\lfloor \frac{k+2}{2} \right\rfloor \right) \\ &\leq \frac{1}{2} \left(\left\lfloor \frac{k+1}{2} + \frac{k+2}{2} \right\rfloor \right) = \frac{1}{2} \left(\left\lfloor \frac{k+1+k+2}{2} \right\rfloor \right) \\ &= \frac{\lfloor k+1.5 \rfloor}{2} = \frac{k+1}{2} \end{aligned}$$

Since $\frac{1}{2} \left\lfloor \frac{k+1}{2} \right\rfloor + \frac{1}{2} \left\lceil \frac{k+1}{2} \right\rceil \leq \frac{1}{2}(k+1) = \frac{k+1}{2}$, that means that it must be less than or equal to exactly half of our $k+1$ items, which is not a majority. So, a majority variant cannot exist without being the majority in one of the sub-problems.

Therefore, for $\text{len}(S) = k + 1$, the algorithm will find the majority variant.

(A loop invariant would normally be necessary, but given how simple the loop is, it is unnecessary. It is obvious that the loop will be able to count the number of times the variant appears. Try proving as an exercise) $\square$